

ATIVIDADE 02

Pergunta 1

Liste todos os alunos cadastrados.

R: select * from aluno

Pergunta 2

Mostre apenas o nome e o curso dos alunos.

R: select nome, curso from aluno

Pergunta 3

Liste os alunos do curso de Computacao.

R: select nome from aluno WHERE curso = 'Computacao'

Pergunta 4

Liste os alunos que moram em Maringa.

R: select nome from aluno WHERE cidade = 'Maringa'

Pergunta 5

Mostre os alunos ordenados pelo nome em ordem alfabética.

R: select nome from aluno ORDER BY NOME

Pergunta 6

Mostre os alunos ordenados pelo ano de ingresso, do mais antigo para o mais recente.

R: select nome from aluno ORDER BY ano_ingresso

Pergunta 7

Liste os alunos que ingressaram a partir de 2022.

R: select nome, ano_ingresso from aluno WHERE ano_ingresso >= 2022

Pergunta 8

Liste os alunos cujo nome começa com a letra A.

R: select nome from aluno WHERE nome LIKE 'A%'

Pergunta 9

Liste os alunos dos cursos Computacao ou Engenharia.

R: select nome from aluno WHERE curso = 'Computacao' OR curso = 'Engenharia'

Pergunta 10

Liste as disciplinas com carga horária entre 60 e 80 horas.

R: select nome from disciplina WHERE carga_horaria > 60 AND carga_horaria < 80

Pergunta 11

Conte quantos alunos existem cadastrados.

R: select COUNT(id) from aluno

Pergunta 12

Calcule a média das notas da tabela matricula.

R: select AVG(NOTA) from matricula

Pergunta 13

Mostre a maior nota registrada.

R: select max(NOTA) from matricula

Pergunta 14

Mostre a menor nota registrada.

R: select min(NOTA) from matricula

Pergunta 15

Calcule a soma das cargas horárias de todas as disciplinas.

R: select SUM(carga_horaria) from disciplina

Pergunta 16

Mostre a quantidade de alunos por curso.

R: select CURSO, COUNT(id) from aluno GROUP BY CURSO

Pergunta 17

Mostre a quantidade de alunos por cidade.

R: select cidade, COUNT(id) from aluno GROUP BY cidade

Pergunta 18

Mostre a média das notas por situação da matrícula.

R: select situacao, avg(nota) from MATRICULA GROUP BY situacao

Pergunta 19

Mostre quantas matrículas existem por semestre.

R: select semestre, COUNT(ID) from MATRICULA GROUP BY semestre

Pergunta 20

Mostre os cursos que possuem mais de 1 aluno cadastrado.

R: select curso from aluno GROUP BY CURSO HAVING COUNT(id) > 1;

Pergunta 21

Liste o nome dos alunos e a situação de suas matrículas.

R: SELECT nome, situacao
FROM aluno JOIN matricula
ON aluno.id = matricula.aluno_id;

Pergunta 22

Liste o nome dos alunos e o nome das disciplinas em que estão matriculados.

R: SELECT a.nome, d.nome
FROM aluno a JOIN disciplina d JOIN matricula m
ON m.aluno_id = a.id AND m.disciplina_id = d.id

Pergunta 23

Liste o nome do aluno, o nome da disciplina e a nota.

R: SELECT a.nome, d.nome, m.nota
FROM aluno a JOIN disciplina d JOIN matricula m
ON m.aluno_id = a.id AND m.disciplina_id = [d.id](#)

Pergunta 24

Liste apenas os alunos matriculados em disciplinas do departamento Computacao.

R: SELECT a.nome, d.nome
FROM aluno a JOIN disciplina d JOIN matricula m
ON m.aluno_id = a.id AND m.disciplina_id = d.id
WHERE departamento= "computacao"

Pergunta 25

Mostre o nome dos alunos que tiveram matrícula com situação Reprovado.

R: SELECT a.nome
FROM matricula m
JOIN aluno a ON m.aluno_id = a.id
WHERE m.situacao = 'Reprovado';

Pergunta 26

Mostre o nome dos alunos de Computacao e as disciplinas que eles cursaram.

R: SELECT a.nome AS aluno,
d.nome AS disciplina
FROM aluno a
JOIN matricula m ON a.id = m.aluno_id
JOIN disciplina d ON d.id = m.disciplina_id
WHERE a.curso = 'Computacao';

Pergunta 27

Mostre a média de notas por aluno.

```
R: SELECT a.nome AS aluno, AVG(m.nota) AS media_notas
FROM matricula m JOIN aluno a ON a.id = m.aluno_id
GROUP BY a.id, a.nome;
```

Pergunta 28

Mostre a quantidade de disciplinas cursadas por cada aluno.

```
R: SELECT a.nome AS aluno, COUNT(m.disciplina_id) AS qtd_disciplinas
FROM matricula m JOIN aluno a ON a.id = m.aluno_id
GROUP BY a.id, a.nome;
```

Pergunta 29

Liste os alunos cuja média de notas foi maior que 8.

```
R: SELECT a.nome
FROM aluno a
JOIN matricula m ON a.id = m.aluno_id
GROUP BY a.id, a.nome
HAVING AVG(m.nota) > 8
```

Pergunta 30

Mostre o departamento e a quantidade de matrículas em disciplinas de cada departamento.

```
R: SELECT d.departamento, COUNT(*) AS qtd_matriculas
FROM matricula m JOIN disciplina d ON d.id = m.disciplina_id
GROUP BY d.departamento;
```

Atividade 03

Primeira Tabela

id	titular	saldo	
 Filter...	 Filter...	 Filter...	
1	Ana	1000.00	
2	Bruno	500.00	
3	Carlos	300.00	
4	Daniela	800.00	

Pergunta 1

Qual é o objetivo da tabela contas neste cenário prático?

R: O objetivo é simular um sistema simples de contas bancárias.

Pergunta 2

Quais são os saldos iniciais de cada titular antes da execução das transações?

R: Os saldos de cada titular antes da execução são:

- Ana: 1000
- Bruno: 500
- Carlos: 300
- Daniela: 800

Segunda Tabela

id	titular	saldo
abc Filter...	abc Filter...	abc Filter...
1	Ana	900.00
2	Bruno	600.00
3	Carlos	300.00
4	Daniela	800.00

Pergunta 3

O que aconteceu com os saldos após o COMMIT?

R: Após o “commit”, o valor referente ao id=1 diminuiu em 100 e o id=2 aumentou em 100. Ou seja, o COMMIT efetivou a transferência de valor entre as contas, salvando as mudanças de forma definitiva.

Pergunta 4

Por que as duas instruções UPDATE devem fazer parte da mesma transação?

R: As duas instruções precisam estar na mesma transação para garantir a consistência dos dados. Nesse caso, elas representam uma única operação lógica: uma transferência de R\$100 da conta da Ana para a do Bruno.

Se cada UPDATE fosse executado separadamente, poderia ocorrer um problema como:

- O valor ser descontado da Ana (primeiro UPDATE executa com sucesso)
- Mas não ser creditado ao Bruno (segundo UPDATE falha, por erro ou interrupção)

Isso causaria perda de dinheiro no sistema, deixando os dados inconsistentes.

Terceira Tabela:

id	titular	saldo
 Filter...	 Filter...	 Filter...
1	Ana	900.00
2	Bruno	600.00
3	Carlos	300.00
4	Daniela	800.00

Pergunta 5

Por que os valores não foram alterados ao final?

R: Os valores não foram alterados porque foi executado o comando ROLLBACK.

O ROLLBACK desfaz todas as operações realizadas dentro da transação desde o START TRANSACTION.

Pergunta 6

Em quais situações reais o uso de ROLLBACK seria essencial?

R: O uso de rollback (em bancos de dados e sistemas transacionais) é essencial sempre que você precisa desfazer operações que não podem ser concluídas com segurança ou consistência. Ele garante que o sistema volte a um estado anterior válido.

Situações:

- Falha em transações financeiras
- Compras online (o estoque é reduzido e o pagamento falha, tem que haver uma forma de restaurar o estoque automaticamente).

Quarta Tabela:

Antes do *rollback*

id	titular	saldo	
abc Filter...	abc Filter...	abc Filter...	
3	Carlos	-1700.00	

Depois do *rollback*

id	titular	saldo	
abc Filter...	abc Filter...	abc Filter...	
3	Carlos	300.00	

Pergunta 7

Por que a transação foi desfeita neste caso?

R: Devido ao uso do comando ROLLBACK, o comando de subtrair 2000 do saldo foi “desfeito”, mantendo o saldo com o valor armazenado antes da transação.

Pergunta 8

Qual problema de integridade poderia ocorrer se essa transação fosse confirmada?

R: Inconsistência dos dados e dos valores.

Quinta Tabela:

id	titular	saldo	
abc Filter...	abc Filter...	abc Filter...	
1	Ana	960.00	
2	Bruno	640.00	
3	Carlos	300.00	
4	Daniela	700.00	

Pergunta 9

Qual conta foi debitada e quais contas foram creditadas?

R: As contas com id 1 e 2 foram creditadas e a conta com id 4 foi debitada. A conta pertencente ao id 3 também havia sido debitada antes da execução do comando ROLLBACK.

Pergunta 10

Por que esse conjunto de operações também deve ser tratado como uma única transação?

R: Para haver consistência de dados.

Sexta Tabela:

Antes do *rollback*

id	titular	saldo	
abc Filter...	abc Filter...	abc Filter...	
1	Ana	810.00	

Depois do rollback

id	titular	saldo	
abc Filter...	abc Filter...	abc Filter...	
1	Ana	960.00	

Pergunta 11

Qual era o objetivo de observar o valor da conta em outra sessão antes do COMMIT?

R: Verificar se outra sessão consegue ver alterações não confirmadas, demonstrando o nível de isolamento da transação.

Pergunta 12

Como esse teste se relaciona com o conceito de isolamento?

R: O isolamento garante que:

- Operações dentro de uma transação não sejam visíveis para outras transações até que sejam confirmadas com COMMIT.
- Cada transação “enxergue” um estado consistente do banco de dados.

Nesse contexto, o teste demonstra como o banco controla a visibilidade das alterações entre transações simultâneas, que é exatamente o papel do isolamento.

Sétima Tabela:

id	titular	saldo
abc Filter...	abc Filter...	abc Filter...
1	Ana	660.00

id	titular	saldo
abc Filter...	abc Filter...	abc Filter...
1	Ana	960.00

Pergunta 13

O que aconteceu com a segunda transação?

R: Ela ficou bloqueada (em espera) até a primeira transação fazer o COMMIT. Depois disso, ela continuou e executou o UPDATE.

Pergunta 14

Por que ela precisou esperar?

R: A segunda transação precisou esperar porque a primeira já havia adquirido um bloqueio exclusivo (lock) sobre o registro com id = 1. Depois disso, ao fazer o UPDATE, o bloqueio continua ativo até o COMMIT ou ROLLBACK. O banco percebe que aquela mesma linha já está bloqueada por outra transação em andamento. Para garantir consistência dos dados e evitar conflitos (como atualizações simultâneas inconsistentes), ele faz a segunda transação esperar até que o bloqueio seja liberado.

Pergunta 15

Qual a função do FOR UPDATE?

R: O FOR UPDATE, bloqueia as linhas selecionadas até que o COMMIT ou ROLLBACK sejam executados, a fim de manter a persistência dos dados.

Oitava Tabela: agr ta certo

id	titular	saldo	
abc Filter...	abc Filter...	abc Filter...	
1	Ana	950.00	
2	Bruno	500.00	
3	Carlos	300.00	
4	Daniela	940.00	

Pergunta 16

Por que nesse caso as transações tendem a não disputar o mesmo recurso?

R: A transação 1 bloqueia apenas a linha onde id = 1 e a transação 2 bloqueia apenas a linha onde id = 4. Como são registros distintos, não há conflito

direto de locks, então as transações conseguem executar em paralelo sem disputar o mesmo recurso.

Pergunta 17

O que esse teste mostra sobre concorrência em linhas diferentes da tabela?

R: Apesar de serem linhas diferentes e que supostamente não têm interferência uma sobre a outra, ao rodar o código antes de dar o COMMIT, a outra tabela permanece bloqueada para visualização, contudo, após realizarmos o COMMIT da primeira tabela e tentarmos executar a segunda ela irá mostrar o resultado das duas execuções que testamos.

Nona Tabela:

id	conta_origem	conta_destino	valor	data_moviment...
abc Filter...	abc Filter...	abc Filter...	abc Filter...	abc Filter...
No data				

Pergunta 18

Qual é a importância de registrar movimentações além de atualizar os saldos?

R: É importante para guardar o histórico de atualizações.

Décima Tabela:

id	titular	saldo	
abc Filter...	abc Filter...	abc Filter...	
1	Ana	950.00	
2	Bruno	380.00	
3	Carlos	420.00	
4	Daniela	940.00	

id	conta_origem	conta_destino	valor	data_moviment...
abc Filter...	abc Filter...	abc Filter...	abc Filter...	abc Filter...
1	2	3	120.00	2026-04-24 13:36:47

Pergunta 19

Por que o INSERT na tabela movimentacoes deve estar na mesma transação dos UPDATES?

R: Para garantir a consistência dos dados, é necessário ter os UPDATES juntamente com o INSERT nas movimentações, para que ao realizar a transação, o programa saiba o valor real em cada conta, e verificar se a transação poderá ser feita.

Pergunta 20

O que poderia acontecer se o histórico fosse gravado, mas os saldos não fossem atualizados, ou vice-versa?

R: Iria gerar inconsistência nos dados.

Décima primeira Tabela:

id	titular	saldo	
 Filter...	 Filter...	 Filter...	
1	Ana	950.00	
2	Bruno	380.00	
3	Carlos	420.00	
4	Daniela	940.00	

id	conta_origem	conta_destino	valor	data_moviment...
 Filter...	 Filter...	 Filter...	 Filter...	 Filter...
1	2	3	120.00	2026-04-24 13:36:47

Pergunta 21

O que o ROLLBACK garantiu nesse cenário?

R: Após as alterações feitas e a implementação do ROLLBACK, a execução do código não mostrou nenhuma alteração, pois a ação foi desfeita.

Pergunta 22

Como esse teste demonstra a propriedade de atomicidade?

R: Ela mostra que ou tudo acontece ou nada acontece. Então, após o rollback, nenhuma tabela mudou.

Décima segunda Tabela:

id	titular	saldo	
abc Filter...	abc Filter...	abc Filter...	
1	Ana	950.00	
2	Bruno	380.00	
3	Carlos	420.00	
4	Daniela	940.00	

id	conta_origem	conta_destino	valor	data_moviment...
abc Filter...	abc Filter...	abc Filter...	abc Filter...	abc Filter...
1	2	3	120.00	2026-04-24 13:36:47

Pergunta 23

Como verificar se o banco permaneceu consistente após todas as operações realizadas?

R: É necessário realizar várias operações, para garantir a consistência dos dados, você tem que ver ou se as operações foram realizadas (COMMIT) ou nada foi alterado (ROLLBACK).

Pergunta 24

Por que a consistência do banco depende não apenas dos comandos SQL, mas também da forma como eles são agrupados em transações?

R: Os dados raramente são alterados por uma única instrução. Em sistemas reais, uma operação lógica (como uma transferência bancária ou um pedido de compra) envolve várias etapas, por isso precisamos das transações. Se você executa comandos SQL separadamente (fora de uma transação), pode acabar com estados intermediários inválidos. Sem transação, o banco fica inconsistente. Já ao agrupar esses comandos em uma transação, o sistema garante que todas as etapas sejam concluídas com sucesso, ou nenhuma delas seja aplicada.

Pergunta 25

Explique o que é uma transação em banco de dados.

R: Uma transação em banco de dados é um conjunto de operações realizadas como uma única unidade lógica de trabalho. Essas operações podem incluir comandos como inserir, atualizar, excluir ou consultar dados.

Pergunta 26

Descreva a diferença entre COMMIT e ROLLBACK.

R: O commit confirma as transações feitas na transação e o rollback desfaz essas alterações.

Pergunta 27

Explique por que uma transferência bancária deve ser tratada como transação.

R: Uma transferência bancária deve ser tratada como uma transação porque envolve várias operações que precisam acontecer juntas para garantir a consistência dos dados. Ao transferir dinheiro, o sistema retira o valor de uma conta e adiciona em outra. Se ocorrer algum erro durante o processo, as alterações precisam ser desfeitas para evitar inconsistências, como o dinheiro sair de uma conta e não chegar à outra. Por isso, a transação garante que todas as operações sejam concluídas com sucesso ou que nenhuma delas seja aplicada.

Pergunta 28

O que pode acontecer se duas transações alterarem o mesmo dado ao mesmo tempo sem controle da concorrência?

R: Se duas transações alterarem o mesmo dado ao mesmo tempo sem controle de concorrência, podem ocorrer inconsistências e perda de informações no banco de dados.

Pergunta 29

Qual a relação entre transações e as propriedades ACID?

R: A transação é uma unidade lógica dentro de um banco de dados. Possui 4 principais propriedades, sendo conhecidas como propriedades ACID: Atomicidade, Consistência, Isolamento, Durabilidade.

A atomicidade é sobre a unidade atômica de processamento. Consistência diz respeito ao banco de dados permanecer consistente após uma transação ser executada. Isolamento trata as transações de forma que a mesma estivesse isolada das demais. Por fim, a durabilidade considera que todas as alterações feitas por uma transação devem ser persistidas no banco de dados.

Pergunta 30

Explique o significado da propriedade de atomicidade no contexto de uma operação bancária.

R: A propriedade de atomicidade significa que uma transação deve ser executada completamente ou não ser executada. No contexto de uma operação bancária isso garante que o dinheiro só será transferido se todas as etapas

ocorrerem corretamente. Se houver falha em qualquer parte do processo, todas as alterações realizadas são desfeitas, evitando inconsistências nos saldos das contas.

Pergunta 31

Explique o que significa dizer que uma transação preserva a consistência do banco de dados.

R: Significa que após a realização de uma operação o banco continuará consistente, ou seja, não deve haver conflitos no banco nem informações incoerentes após uma transação, os dados devem respeitar as restrições do banco.

Pergunta 32

Descreva o papel do isolamento em ambientes com múltiplos usuários acessando o mesmo banco.

R: O isolamento garante que transações executadas ao mesmo tempo por múltiplos usuários não interfiram umas nas outras. Cada transação é tratada como se estivesse sendo executada sozinha.

Pergunta 33

Explique a importância da durabilidade após a execução de um **COMMIT**.

R: Após a execução de um **COMMIT**, a transação é executada. Sendo assim, a durabilidade garante que todas essas alterações que foram feitas conforme a transação foi realizada, irão persistir no banco de dados.

Pergunta 34

O que é controle de concorrência e por que ele é necessário?

R: O controle de concorrência é um mecanismo que permite que várias transações aconteçam ao mesmo tempo sem causar erros ou inconsistências nos dados. Ele é necessário para garantir a integridade e a confiabilidade das informações.

Pergunta 35

Explique a função do lock em transações concorrentes.

R: A função do lock nessas operações é controlar o acesso simultâneo das informações para não gerar inconsistências no banco de dados.

Pergunta 36

Descreva um exemplo prático em que o **FOR UPDATE** seja necessário.

R: Um exemplo prático do uso de FOR UPDATE ocorre em uma transferência bancária. Quando o sistema consulta o saldo de uma conta para realizar a transferência, é importante bloquear aquele registro para que outra transação não altere o saldo ao mesmo tempo. Assim, o comando **SELECT ... FOR UPDATE** bloqueia temporariamente a linha selecionada até o fim da transação, evitando problemas como duas transferências utilizarem o mesmo saldo simultaneamente e causando inconsistências nos dados.

Pergunta 37

O que é uma atualização perdida (*lost update*)?

R: Uma atualização perdida é um problema de concorrência que ocorre quando duas ou mais transações tentam alterar o mesmo dado em um banco de dados simultaneamente.

Pergunta 38

Explique por que nem toda leitura concorrente gera problema, mas algumas atualizações simultâneas sim.

R: Pois quando executado a leitura eles não interferem nos valores dos dados, então realizar diversas leituras ao mesmo tempo não gera inconsistência. Já as atualizações podem gerar inconsistência, pois duas transações podem modificar o mesmo dado ao mesmo tempo.

Pergunta 39

Qual é a importância de registrar operações em uma tabela de histórico dentro da mesma transação?

R: É importante para garantir que os dados principais do histórico fiquem sempre sincronizados e consistentes, pois se uma operação é efetiva o histórico também é, caso contrário nenhum dos dois são salvos.

Pergunta 40

Em um sistema acadêmico, cite um exemplo de operação que deveria ser tratada como transação.

R: Em um sistema acadêmico, a matrícula de um aluno em uma disciplina deve ser tratada como uma transação.

Pergunta 41

Em um sistema de estoque, cite um exemplo de falha que poderia justificar o uso de **ROLLBACK**.

R: Uma falha que justificaria o uso do **ROLLBACK** seria um problema na hora de efetuar o pagamento. Como o pagamento falhou, a operação lógica de "venda" não pode ser concluída com sucesso. Para manter a consistência do banco de dados e evitar que o estoque fique incorreto, ou seja, reduzido mas sem a venda correspondente, o comando **ROLLBACK** é essencial. Ele desfaz a redução do estoque que já havia sido feita, restaurando o sistema ao estado anterior válido antes da tentativa de venda.

Pergunta 42

Como o processamento de transações contribui para a confiabilidade de sistemas de informação?

R: O processamento de transações contribui para a confiabilidade dos sistemas de informação porque garante que as operações no banco de dados sejam executadas de forma segura, consistente e previsível.

Pergunta 43

Considerando todos os experimentos realizados, explique de forma integrada como a atomicidade, consistência, isolamento e durabilidade atuam em conjunto no processamento de transações.

R: As quatro propriedades ACID atuam em conjunto para garantir à segurança, confiabilidade e correção, evitando falhas mesmo quando são realizadas diversas operações e se tem muitos usuários acessando o banco ao mesmo tempo.

Desafio

Pergunta 44

Adapte o exemplo bancário para um sistema de matrícula em disciplinas, em que uma transação deva:

- verificar vaga disponível
- reduzir a quantidade de vagas
- registrar a matrícula do aluno

Explique por que essas operações devem ocorrer na mesma transação.

R: Essas operações devem ser realizadas na mesma transação para garantir a consistência do sistema. Assim, a matrícula só será efetuada se tiverem vagas disponíveis, a quantidade de vagas for calculada corretamente e o registro do aluno for efetuado corretamente.

Pergunta 45

Adapte o exemplo para um sistema de estoque e vendas, explicando quais operações devem ser agrupadas para evitar inconsistências.

R: Em um sistema de estoque e vendas, uma transação poderia funcionar da seguinte forma: primeiro o sistema verifica se o produto possui quantidade disponível em estoque, depois reduz a quantidade do produto vendido e por fim registra a venda no histórico do sistema. Essas operações devem ocorrer na mesma transação para garantir a consistência dos dados, evitando situações em que a venda seja registrada sem atualizar o estoque ou o estoque seja alterado sem que a venda seja registrada corretamente.

Atividade 04

id	titular	saldo
1	Ana	1000.00
2	Bruno	500.00
3	Carlos	300.00
4	Daniela	800.00
NULL	NULL	NULL

Pergunta 1

Qual é a finalidade de manter dados iniciais conhecidos antes dos testes de concorrência?

R: A finalidade de manter dados iniciais conhecidos antes dos testes de concorrência é permitir que os resultados das transações possam ser analisados corretamente. Assim, é possível comparar os valores antes e depois das operações e verificar se o comportamento do banco ocorreu como esperado.

Pergunta 2

Por que é importante que a tabela esteja em um estado consistente antes do início dos experimentos?

R: É importante que a tabela esteja em um estado consistente antes do início dos experimentos para garantir que os testes sejam confiáveis e não sofram influência de erros ou alterações anteriores. Dessa forma, os resultados obtidos refletem apenas o efeito das transações executadas durante os testes.

id	titular	saldo
1	Ana	950.00
2	Bruno	500.00
3	Carlos	300.00
4	Daniela	800.00
NULL	NULL	NULL

Pergunta 3

O que aconteceu com a operação realizada na Sessão 2?

R: A operação realizada na Sessão 2 ficou bloqueada temporariamente enquanto a Sessão 1 ainda estava utilizando o registro da conta de id 1. Após o COMMIT da primeira sessão, a segunda transação pôde continuar sua execução normalmente.

Pergunta 4

Por que a segunda sessão precisou aguardar?

R: A segunda sessão precisou aguardar porque a primeira transação havia bloqueado o registro da conta utilizando FOR UPDATE. Isso impede que duas transações alterem o mesmo dado ao mesmo tempo, evitando inconsistências no banco.

Pergunta 5

Qual é a função do comando **FOR UPDATE** nesse experimento?

R: O comando FOR UPDATE tem a função de bloquear as linhas selecionadas durante a transação, impedindo alterações por outras sessões até que a transação atual seja finalizada com COMMIT ou ROLLBACK. Isso garante maior controle e segurança nas operações concorrentes.

id	titular	saldo
1	Ana	900.00
2	Bruno	500.00
3	Carlos	300.00
4	Daniela	870.00
NULL	NULL	NULL

Pergunta 6

Por que, nesse caso, as duas transações tendem a coexistir sem espera significativa?

R: Nesse caso, as duas transações tendem a coexistir sem espera significativa porque cada uma está alterando um registro diferente da tabela. Como

não há acesso concorrente à mesma linha, o banco consegue executar as operações simultaneamente sem gerar bloqueios entre as sessões.

Pergunta 7

O que esse comportamento revela sobre bloqueios em nível de linha?

R: Esse comportamento revela que os bloqueios ocorrem em nível de linha, ou seja, apenas os registros modificados ficam bloqueados. Assim, outras transações podem acessar e alterar linhas diferentes da mesma tabela ao mesmo tempo sem causar conflitos.

id	titular	saldo
1	Ana	900.00
2	Bruno	500.00
3	Carlos	300.00
4	Daniela	870.00
NULL	NULL	NULL

Pergunta 8

Qual era o objetivo de consultar o mesmo registro em outra sessão antes do COMMIT?

R: O objetivo de consultar o mesmo registro em outra sessão antes do COMMIT era verificar se uma transação não finalizada teria suas alterações visíveis para outras sessões do banco de dados.

Pergunta 9

Como esse experimento se relaciona com o conceito de isolamento?

R: Esse experimento se relaciona com o conceito de isolamento porque mostra que alterações feitas dentro de uma transação ainda não confirmada não devem ficar visíveis para outras sessões. Isso garante que cada transação seja executada de forma independente, evitando inconsistências nos dados.

id	titular	saldo
3	Carlos	400.00
NULL	NULL	NULL

Pergunta 10

O valor lido na Sessão 1 permaneceu o mesmo ou mudou?

R: O valor lido na Sessão 1 pode permanecer o mesmo ou mudar, dependendo do nível de isolamento utilizado pelo banco de dados. Em muitos casos, a segunda leitura já mostrará o valor atualizado após o COMMIT da Sessão 2.

Pergunta 11

Que tipo de fenômeno esse teste procura identificar?

R: Esse teste procura identificar o fenômeno chamado de leitura não repetível (non-repeatable read), que ocorre quando uma mesma consulta retorna resultados diferentes dentro da mesma transação porque outra transação modificou e confirmou os dados nesse intervalo.

id	titular	saldo
4	Daniela	570.00
NULL	NULL	NULL

Pergunta 12

Por que operações concorrentes sobre o mesmo registro exigem maior controle?

R: Operações concorrentes sobre o mesmo registro exigem maior controle porque podem acessar e modificar o mesmo dado ao mesmo tempo, causando conflitos de leitura e escrita. Sem controle adequado, uma transação pode sobrescrever a outra ou trabalhar com valores desatualizados, comprometendo a consistência do banco de dados.

Pergunta 13

Que inconsistência pode surgir quando duas transações tentam atualizar o mesmo dado quase ao mesmo tempo?

R: A principal inconsistência que pode surgir é a perda de atualização (lost update), onde uma transação sobrescreve a alteração feita por outra. Isso pode fazer com que uma das reduções de saldo não seja considerada corretamente, resultando em um valor final incorreto e inconsistente no banco.

id	titular	saldo
1	Ana	900.00
2	Bruno	510.00
3	Carlos	400.00
4	Daniela	570.00
NULL	NULL	NULL

Pergunta 14

Qual evidência mostra que havia um bloqueio ativo sobre o registro?

R: A evidência de que havia um bloqueio ativo sobre o registro é que a Sessão 2 ficou aguardando a execução do UPDATE, sem conseguir concluir a operação. Isso ocorre porque o SELECT com o FOR UPDATE na Sessão 1 bloqueou a linha da conta de id 2, impedindo que outra transação a modificasse até o término da primeira.

Pergunta 15

Por que a liberação do lock depende do fim da transação?

R: A liberação do lock depende do fim da transação porque o banco de dados precisa garantir a integridade e a consistência dos dados. Enquanto a transação não é finalizada com COMMIT ou ROLLBACK, o sistema mantém o bloqueio para evitar que outras transações alterem ou leiam dados intermediários que ainda não foram confirmados.

id	titular	saldo
1	Ana	900.00
2	Bruno	510.00
3	Carlos	400.00
4	Daniela	570.00
NULL	NULL	NULL

Pergunta 16

Por que a segunda leitura com **FOR UPDATE** não pôde prosseguir imediatamente?

R: A segunda leitura com FOR UPDATE não pôde prosseguir imediatamente porque a primeira sessão já havia bloqueado o registro da conta de id 1. Como esse comando coloca um bloqueio na linha selecionada, a Sessão 2 precisa esperar a liberação desse lock antes de continuar, o que só acontece após o COMMIT da Sessão 1.

Pergunta 17

Em que essa situação difere de uma consulta **SELECT** comum?

R: Essa situação difere de uma consulta SELECT comum porque o SELECT normal apenas lê os dados sem bloquear o registro, permitindo que outras sessões também leiam ou até atualizem o mesmo dado simultaneamente. Já o SELECT com o FOR UPDATE bloqueia a linha, impedindo alterações concorrentes até o fim da transação, garantindo controle sobre atualizações simultâneas.

Pergunta 18

Qual seria o saldo correto ao final, caso ambas as operações fossem consideradas corretamente?

R: O saldo correto ao final deveria ser 700, pois as duas operações devem ser aplicadas sobre o mesmo valor inicial de 1000: $1000 - 100 - 200 = 700$.

Pergunta 19

Por que o resultado 800 caracteriza uma atualização perdida?

R: O resultado 800 caracteriza uma atualização perdida porque uma das transações sobrescreveu a outra sem considerar sua alteração. A Transação B leu o valor inicial (1000) e gravou 800, ignorando a redução feita pela Transação A. Assim, a alteração de A (redução de 100) foi perdida, gerando inconsistência nos dados.

id	descricao
1	Operacao realizada pela sessao 1
2	Operacao realizada pela sessao 2
NULL	NULL

Pergunta 20

Por que inserções em linhas diferentes nem sempre geram conflito direto?

R: Inserções em linhas diferentes nem sempre geram conflito direto porque cada INSERT cria um novo registro na tabela, sem precisar alterar ou bloquear dados já existentes. Assim, como não há tentativa de modificar o mesmo registro, as transações podem ser executadas simultaneamente sem interferência direta.

Pergunta 21

O que esse experimento mostra sobre concorrência quando não há disputa pelo mesmo registro?

R: Esse experimento mostra que, quando não há disputa pelo mesmo registro, o banco de dados consegue executar transações concorrentes ao mesmo tempo com eficiência. Isso evidencia que a concorrência é permitida em nível de tabela, desde que as operações não afetem os mesmos dados, garantindo melhor desempenho sem comprometer a integridade.

id	titular	saldo
1	Ana	900.00
2	Bruno	510.00
3	Carlos	420.00
4	Daniela	570.00
NULL	NULL	NULL

Pergunta 22

Quais impactos um bloqueio mantido por muito tempo pode causar em um sistema real?

R: Um bloqueio mantido por muito tempo pode causar lentidão no sistema, filas de espera em outras transações e até indisponibilidade de operações, já que outras sessões ficam impedidas de acessar ou atualizar os dados bloqueados.

Pergunta 23

Por que transações longas tendem a ser indesejáveis em ambientes concorrentes?

R: Transações longas são indesejáveis em ambientes concorrentes porque aumentam o tempo de bloqueio dos dados, reduzindo o desempenho do sistema e aumentando a chance de conflitos, espera excessiva e degradação da performance geral do banco de dados.

	id	titular	saldo
▶	1	Ana	900.00
	2	Bruno	510.00
	3	Carlos	420.00
	4	Daniela	570.00
▲	NULL	NULL	NULL

	id	descricao
	1	Operacao realizada pela sessao 1
	2	Operacao realizada pela sessao 2
	NULL	NULL

Pergunta 24

Como verificar se o banco permaneceu consistente após todos os cenários executados?

R: Para verificar se o banco permaneceu consistente, deve-se analisar se os valores finais das contas correspondem exatamente às operações realizadas durante os testes e se não houve perdas, duplicações ou alterações incorretas. Também é importante conferir se o histórico na tabela log_operacoes registra corretamente todas as inserções feitas nas transações.

Pergunta 25

Por que a análise final dos dados é importante após testes de concorrência?

R: A análise final dos dados é importante após testes de concorrência porque permite identificar possíveis problemas como inconsistências, atualizações perdidas ou falhas de isolamento entre transações. Ela garante que o banco se comportou corretamente sob execução simultânea e que a integridade dos dados foi mantida em todos os cenários.

Atividade dissertativa

Questão 26

Explique o que é concorrência em banco de dados.

R: É quando duas ou mais transações ou usuários acessam e modificam os mesmos dados ao mesmo tempo.

Questão 27

Descreva o papel dos bloqueios no controle de concorrência.

R: A Técnica Bloqueio é utilizada dentro do protocolo pessimista do controle de concorrência. O protocolo pessimista se baseia na premissa de que os conflitos entre as transações ocorrem com frequência. Os bloqueios dentro do protocolo são utilizados para controlar a execução concorrente das transações.

Bloqueio é uma variável associada a um item do banco de dados que indica o status do mesmo em relação às possíveis operações que podem ocorrer ser aplicadas. Normalmente, existe um bloqueio associado a cada um dos itens de dado do banco de dados, além de serem usados como forma de sincronizar o acesso por transações concorrentes aos itens do banco de dados.

Questão 28

Explique a diferença entre acessar registros iguais e registros diferentes em transações simultâneas.

R: Ao acessar registros iguais em transações simultâneas, pode ser que haja alguma inconsistência no sistema, ao acessar registros diferentes, eles não interferem entre si.

Questão 29

Por que **FOR UPDATE** é importante em determinadas operações críticas?

R: O comando **FOR UPDATE** é importante em operações críticas de banco de dados uma vez que, ao ser anexado a uma instrução **SELECT**, ele gera um bloqueio de escrita sobre os registros selecionados e o mantém por toda a duração da transação. Isso é importante para eliminar o problema da Atualização Perdida (Lost Update), garantindo que o dado lido não seja alterado por outra transação antes que a transação atual complete seu ciclo de "ler, calcular e gravar".

Questão 30

O que significa dizer que uma transação ficou esperando outra liberar um recurso?

R: Significa que uma transação não pode continuar naquele momento porque o dado ou recurso que ela precisa está sendo usado por outra transação.

Questão 31

Explique o conceito de atualização perdida.

R: Atualização Perdida é um problema de concorrência que ocorre quando duas transações simultâneas leem o mesmo dado, o modificam e tentam gravar suas alterações, resultando na perda da modificação realizada pela primeira transação que gravou, pois a segunda transação a sobrescreve sem considerar a alteração intermediária.

Questão 32

Descreva por que o isolamento é essencial em sistemas multiusuário.

R: O isolamento é essencial em sistemas multiusuário, uma vez que garante que a execução de transações simultâneas não afete a consistência dos dados, fazendo com que cada transação se comporte como se estivesse sendo executada sozinha, de forma serializada.

Questão 33

Explique como uma leitura pode ser afetada por outra transação ainda não concluída.

R: Uma leitura pode ser afetada por outra transação ainda não concluída, principalmente quando ocorre o problema da Leitura Suja. Este problema ocorre quando uma transação lê um dado modificado por outra transação que ainda não executou o **COMMIT**, o que significa que as alterações lidas são temporárias e podem ser desfeitas por um **ROLLBACK**. Se a transação leitora basear suas decisões nesse dado não confirmado, ela estará utilizando informações inconsistentes ou inválidas, caso a transação modificadora decida desfazer suas alterações.

Questão 34

Por que transações longas podem prejudicar o desempenho de sistemas concorrentes?

R: Transações longas podem prejudicar o desempenho de sistemas concorrentes porque mantêm recursos bloqueados por mais tempo, impedindo que outras transações acessem os mesmos dados.

Questão 35

Qual é a relação entre concorrência e consistência dos dados?

R: A concorrência representa o risco de que a consistência dos dados seja violada. Por outro lado, a consistência é o objetivo que o isolamento busca garantir, fazendo com que cada transação se comporte como se estivesse sendo executada sozinha, de forma serializada. A concorrência gera problemas quando transações simultâneas acessam os mesmos registros, podendo levar a inconsistências como a Atualização Perdida ou a Leitura Suja. Para resolver isso, o mecanismo de bloqueio é usado para gerenciar a concorrência e manter a consistência.

Questão 36

Descreva um exemplo real em que duas transações possam disputar o mesmo dado.

R: Podemos citar como exemplo a gestão do saldo em uma conta bancária, onde duas transações tentam sacar ou transferir dinheiro da mesma conta simultaneamente. Se uma Transação 1 lê o saldo (R\$ 500) e calcula seu saque, e, ao mesmo tempo, uma Transação 2 também lê o mesmo valor original (R\$ 500), ambas baseiam seus cálculos nesse valor. Quando a Transação 1 grava o novo saldo (R\$ 400), a Transação 2, ao gravar seu resultado (R\$ 300), sobrescreve a alteração da T1, resultando em um saldo final incorreto. O dado disputado é o campo do saldo, e o sistema de banco de dados precisaria usar mecanismos de bloqueio, como o **FOR UPDATE**, para forçar a serialização das operações e garantir que a T2 esperasse a T1 concluir, evitando a perda de dados e mantendo a consistência.

Questão 37

Explique por que nem toda operação simultânea gera conflito.

R: Nem toda operação simultânea gera conflito porque o risco de conflitos em transações simultâneas está ligado à disputa pelos mesmos recursos. Quando transações simultâneas acessam registros diferentes, as operações são independentes e não interferem nos dados lidos ou modificados por outras transações.

Questão 38

Como o banco de dados contribui para impedir que alterações simultâneas corrompam os dados?

R: O banco de dados impede que alterações simultâneas corrompam os dados usando mecanismos de controle de concorrência. Esses mecanismos

garantem que várias transações possam ocorrer ao mesmo tempo sem causar inconsistências.

Questão 39

Explique o que aconteceria em um sistema bancário sem mecanismos de lock.

R: Um sistema bancário sem mecanismos de bloqueio falharia em garantir a consistência e a integridade do saldo das contas em um ambiente concorrente. O principal problema seria a Atualização Perdida, onde duas transações que tentassem modificar o mesmo saldo leriam o mesmo valor inicial e a que gravasse por último iria sobrescrever o resultado da primeira transação, fazendo com que o saque ou depósito desta fosse "perdido". Além disso, ocorreria também a Leitura Suja, em que uma transação leria um saldo temporário, modificado por outra transação ainda não confirmada, e que poderia ser desfeito por um **ROLLBACK**, levando a operações baseadas em dados incorretos ou inválidos. Em suma, sem o isolamento forçado pelos bloqueios, as operações financeiras não seriam executadas de forma serializada, resultando em cálculos de saldo incorretos.

Questão 40

Qual a importância de observar a ordem de execução das transações em testes práticos?

R: A importância de observar a ordem de execução das transações em testes práticos é crucial para simular e verificar como o sistema lida com a concorrência e garante o isolamento. A ordem de execução (ou o entrelaçamento das operações) é o que define se ocorrerão conflitos e quebras de consistência, permitindo a identificação de inconsistências como a Atualização Perdida ou a Leitura Suja em cenários onde transações acessam "registros iguais".

9. Questão integradora

Questão 41

Considerando todos os experimentos realizados, explique de forma integrada como concorrência, bloqueios e isolamento atuam juntos para evitar inconsistências no banco de dados.

R: O relacionamento entre concorrência, bloqueios e isolamento é a base do controle transacional em um banco de dados, agindo em conjunto para manter a consistência dos dados. A concorrência representa o risco de violação da consistência quando transações simultâneas acessam registros iguais. Sem controle, isso leva a inconsistências como a Atualização Perdida e a Leitura Suja. O isolamento é o objetivo, garantindo que as transações se comportem como se

fossem executadas de forma serializada. Para alcançar o isolamento, o mecanismo de bloqueio é usado para forçar a serialização das operações. Isso ocorre quando uma transação bloqueadora obtém exclusividade, obrigando outras a "ficar esperando" até que o recurso seja liberado.

10. Desafio adicional

Questão 42

Adapte os testes realizados para um sistema de estoque em que dois usuários tentam vender o mesmo produto simultaneamente. Explique quais riscos existem e como o banco pode evitá-los.

R: Em um sistema de estoque, dois usuários tentando vender o mesmo produto ao mesmo tempo podem gerar problemas como estoque negativo ou venda duplicada. Isso acontece quando ambos leem a quantidade disponível antes da atualização. O banco pode evitar esses problemas usando controle de concorrência, como bloqueio de linha (FOR UPDATE), garantindo que apenas uma transação altere o estoque por vez, além de utilizar transações para manter a consistência.

Questão 43

Adapte os testes para um sistema de matrícula acadêmica, em que duas pessoas tentam ocupar a última vaga da mesma disciplina ao mesmo tempo.

R: Em um sistema de matrícula acadêmica, duas pessoas tentando ocupar a última vaga da mesma disciplina ao mesmo tempo podem causar excesso de matrículas além da capacidade permitida. O banco evita esse problema utilizando bloqueios e transações, garantindo que a verificação de vagas, a redução da disponibilidade e o registro da matrícula ocorram de forma atômica e isolada.

Questão 44

Explique como você organizaria um experimento prático no VS Code com duas sessões para demonstrar espera por lock a outros estudantes.

R: Para organizar um experimento no VS Code com duas sessões, eu abriria duas conexões diferentes com o banco de dados (dois terminais ou abas SQL). Na primeira sessão, iniciaria uma transação e usaria SELECT com o FOR UPDATE para bloquear um registro, mantendo a transação aberta. Na segunda sessão, tentaria atualizar o mesmo registro para mostrar que ela fica em espera até a liberação do lock. Depois, finalizaria a primeira sessão com COMMIT para liberar a segunda. Isso demonstra claramente o funcionamento de bloqueio e espera por lock.

Questão 45

Compare um cenário com controle de concorrência e outro sem controle de concorrência, destacando os impactos sobre a confiabilidade dos dados.

R: Em um cenário com controle de concorrência, o banco garante consistência, evitando conflitos como atualizações perdidas e mantendo a integridade dos dados mesmo com várias transações simultâneas. Já em um cenário sem controle de concorrência, podem ocorrer problemas como dados incorretos, sobrescrita de informações e inconsistências, comprometendo a confiabilidade do sistema.

8. Atividade prática com enunciado formal

Enunciado

Um sistema bancário multiusuário precisa permitir operações simultâneas sem comprometer a integridade dos dados. Para isso, implemente testes em SQL que demonstrem:

- bloqueio explícito de registros com **FOR UPDATE**

```
START TRANSACTION;
```

```
SELECT * FROM contas
```

```
WHERE id = 1
```

```
FOR UPDATE;
```

```
UPDATE contas
```

```
SET saldo = saldo - 100
```

```
WHERE id = 1;
```

- espera de uma transação por outra

```
START TRANSACTION;
```

```
UPDATE contas
```

```
SET saldo = saldo + 50
```

```
WHERE id = 1;
```

- diferença entre concorrência em registros iguais e em registros diferentes

Em registros diferentes: Não há bloqueio entre elas.

-- Sessão 1

UPDATE contas SET saldo = saldo - 50 WHERE id = 1;

-- Sessão 2

UPDATE contas SET saldo = saldo + 70 WHERE id = 4;

Em registro iguais:

Sessão 1:

START TRANSACTION;

SELECT saldo FROM contas WHERE id = 1;

UPDATE contas

SET saldo = saldo - 100

WHERE id = 1;

- Mantém a transação aberta (sem COMMIT ainda)

Sessão 2:

START TRANSACTION;

SELECT saldo FROM contas WHERE id = 1;

UPDATE contas

SET saldo = saldo - 200

WHERE id = 1;

- Fica bloqueada

Finalize:

-- Sessão 1

COMMIT;

-- Sessão 2

COMMIT;

- risco de atualização perdida

Seção 1:

START TRANSACTION;

-- lê valor

SELECT saldo FROM contas WHERE id = 1;

-- calcula e grava

UPDATE contas

SET saldo = 900

WHERE id = 1;

Seção 2:

START TRANSACTION;

-- lê o MESMO valor antigo

SELECT saldo FROM contas WHERE id = 1;

-- sobrescreve sem saber da A

UPDATE contas

SET saldo = 800

WHERE id = 1;

- análise da consistência final dos dados após execuções concorrentes

SELECT * FROM contas;

Objetivos

Ao final da atividade, o estudante deve ser capaz de:

- compreender o conceito de concorrência

- identificar situações de bloqueio
- analisar o efeito de locks em duas sessões simultâneas
- perceber quando há disputa por recursos
- discutir riscos de inconsistência em operações concorrentes
- relacionar concorrência com integridade e desempenho

Tarefa final

Com base nos testes realizados, produza um texto explicando:

1. o que é concorrência em banco de dados

R: Concorrência em banco de dados é a capacidade de múltiplas transações serem executadas ao mesmo tempo, acessando e modificando dados compartilhados. Isso melhora o desempenho do sistema, mas exige controle para evitar inconsistências.

2. como funcionam os locks

R: Locks são mecanismos que impedem que duas transações modifiquem o mesmo dado simultaneamente. Quando usamos FOR UPDATE, o registro fica bloqueado até que a transação seja finalizada com COMMIT ou ROLLBACK.

3. por que algumas transações precisam esperar

R: Uma transação precisa esperar quando outra já está utilizando o mesmo registro. Isso acontece para evitar conflitos de escrita e garantir que os dados não sejam alterados simultaneamente de forma inconsistente.

4. o que é atualização perdida

R: Atualização perdida ocorre quando duas transações leem o mesmo valor e escrevem resultados diferentes, fazendo com que uma sobrescreva a outra. Isso gera perda de uma das atualizações.

5. por que o isolamento é importante

R: O isolamento garante que cada transação seja executada de forma independente, sem interferência de outras transações em andamento. Isso evita leituras inconsistentes e erros de concorrência.

6. como o banco preserva a consistência em acessos simultâneos

R: O banco preserva a consistência por meio de transações, locks, níveis de isolamento e controle de concorrência. Esses mecanismos garantem que os

dados permaneçam corretos mesmo com múltiplos usuários acessando o sistema ao mesmo tempo.